

A wizard, ray-marched

Pictures (and music) from pure functions, in Futhark, on the GPU

For programmers who have never done computer graphics

What we built



A web page showing a wizard on a floating rock, flying over endless land, casting spells, with music.

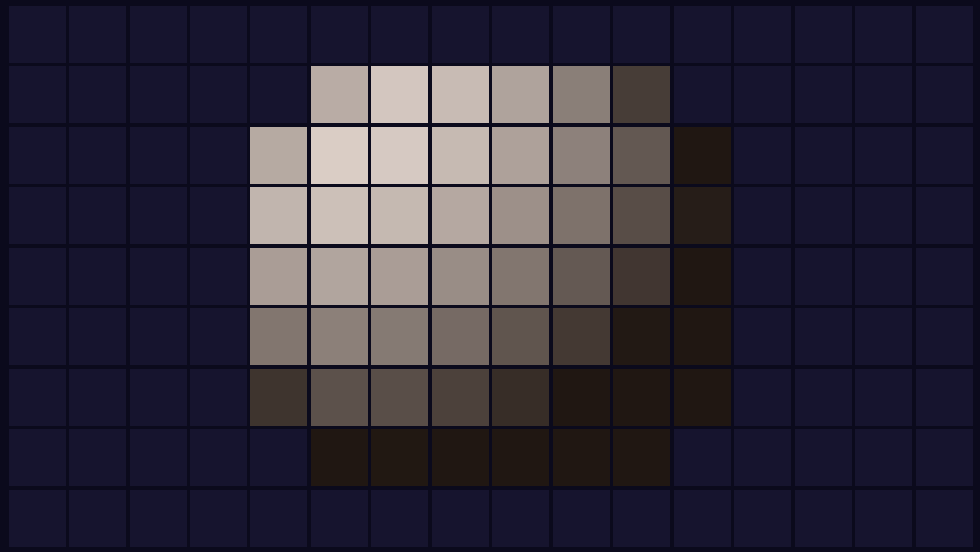
There are no 3D models, no image files and no audio files. Every pixel’s colour, and every sound sample, is computed by a **pure function** from the time, the camera and a few settings.

It runs at 1920×1080, 60 times a second, in a browser.

The basics

How a computer draws a 3D picture

The GPU: one function, millions of times



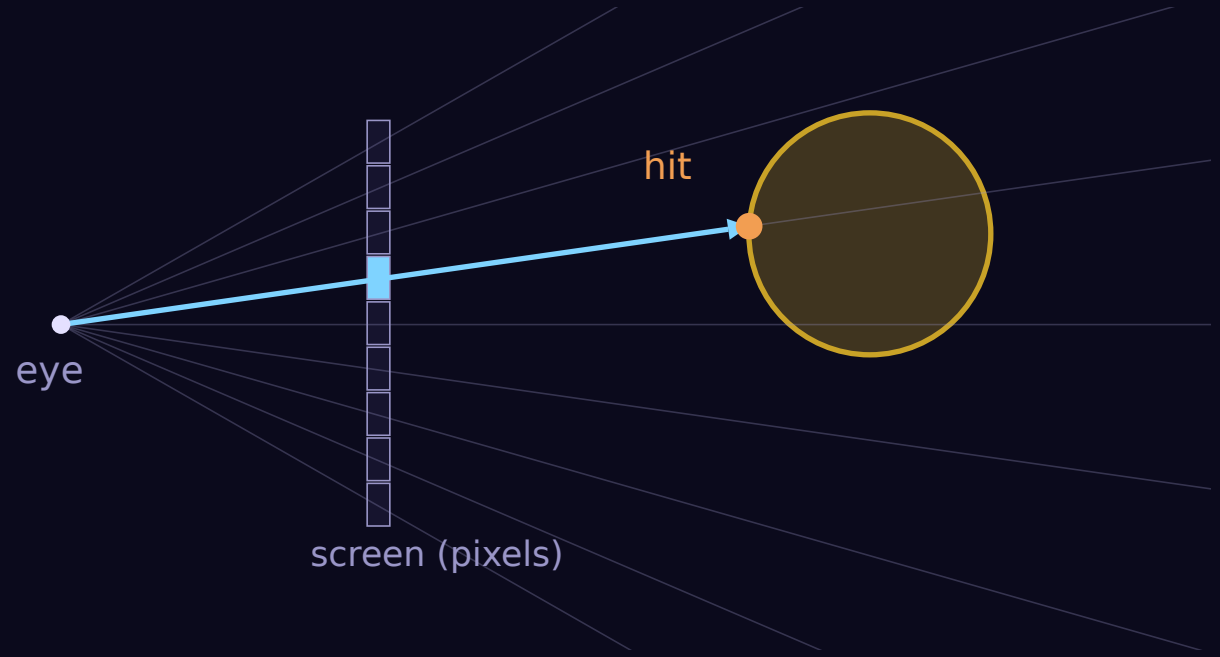
each square: one pixel, one **thread**, the same code

A screen is a grid of **pixels**: 2 million at 1080p, 60 frames a second, so 120 million colours a second.

A **GPU** has thousands of simple cores built for this: they all run **the same function**, each on different input (here, each pixel's position).

That function is called a **shader** or **kernel**. Ours is: *given a pixel, what colour is it?*

A camera made of rays

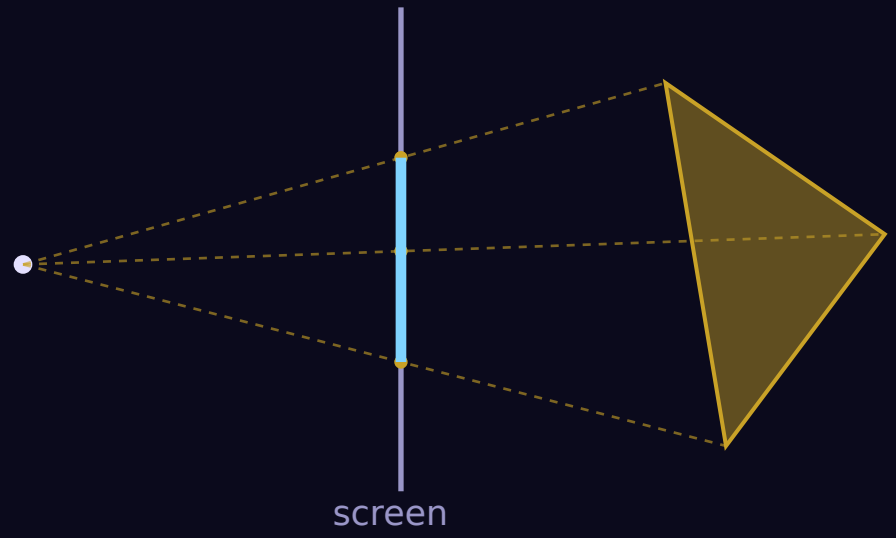


Put the eye at a point and the screen in front of it. For each pixel, shoot a **ray** (a half-line) from the eye through that pixel.

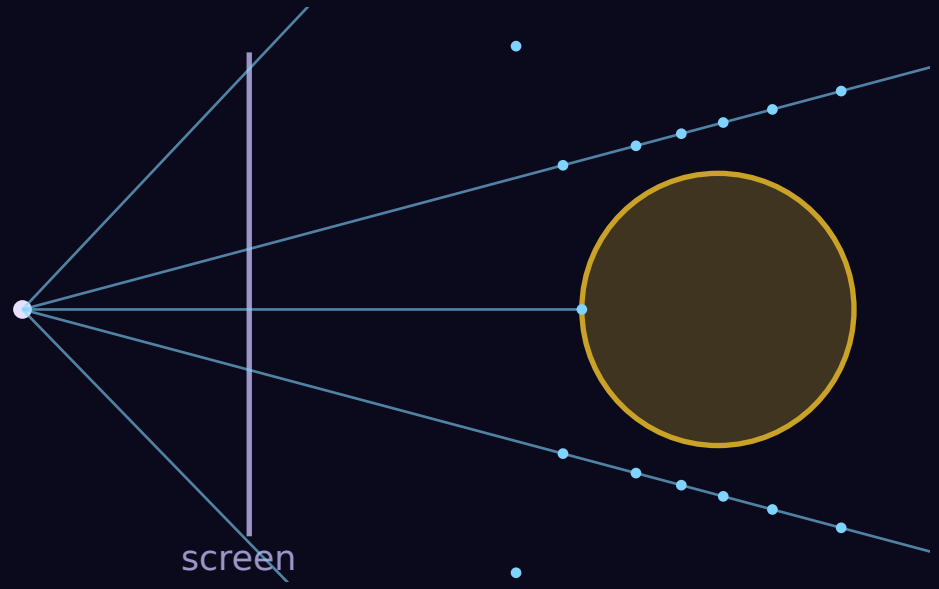
Whatever the ray hits first is what that pixel shows. Its colour depends on **what** it hit and **how that spot is lit**.

So drawing a picture comes down to two questions per pixel: where does the ray hit, and how bright is it there?

Two ways to find what a ray hits



Rasterising (games): shapes are made of triangles. Each triangle’s corners are projected onto the screen and the pixels between them filled in.

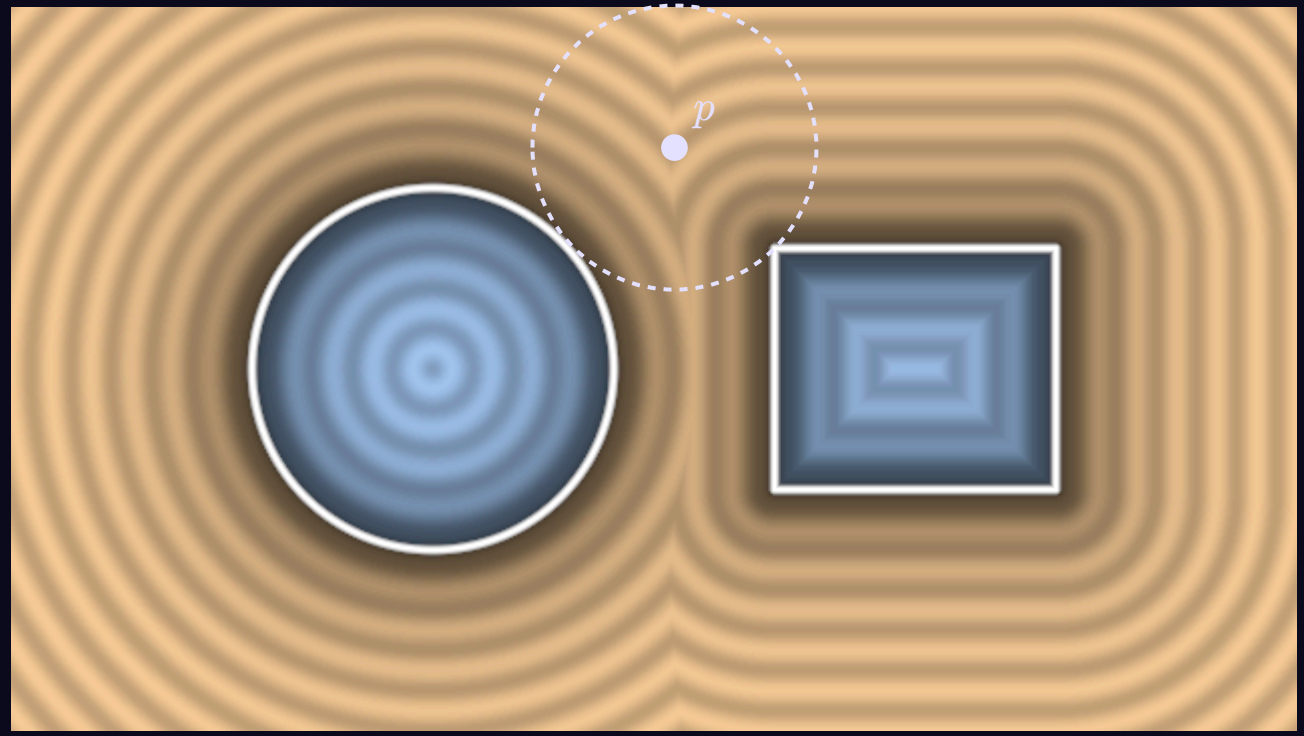


Ray marching (this project): shapes are **functions**. Each ray walks forward in steps, asking the scene “how far am I from the nearest surface?” until the answer is ≈ 0.

Shapes as functions

Distance functions and sphere tracing

A shape is a distance function



Every point coloured by its distance: orange outside, blue inside, a band every 0.1 units, the surface in white. The dashed circle around p has radius $d(p)$: it just touches the nearest surface.

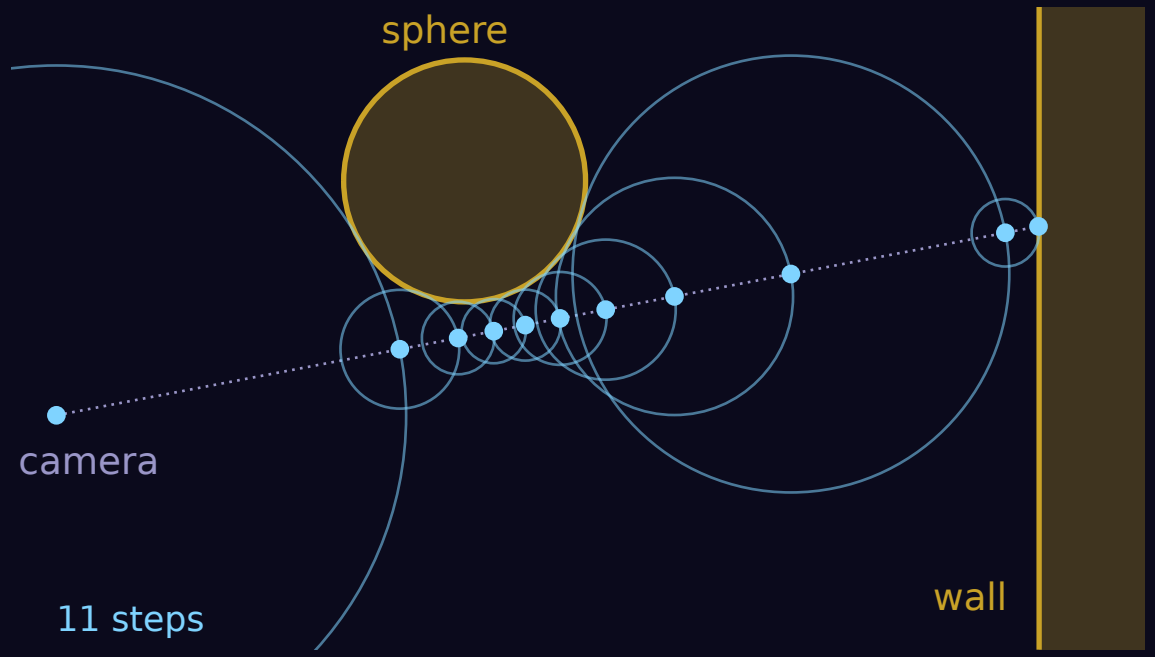
A **signed distance function** (SDF) $d(p)$ takes a point and returns how far it is from the nearest surface: positive outside, negative inside, 0 on it.

```
-- a circle of radius r at the origin
def circle p r = length p - r

-- a scene: the nearer of two shapes
def scene p =
  min (circle (p - c1) 0.45)
      (box (p - c2) size)
```

Nothing else describes the shapes: no lists of points or triangles.

Sphere tracing

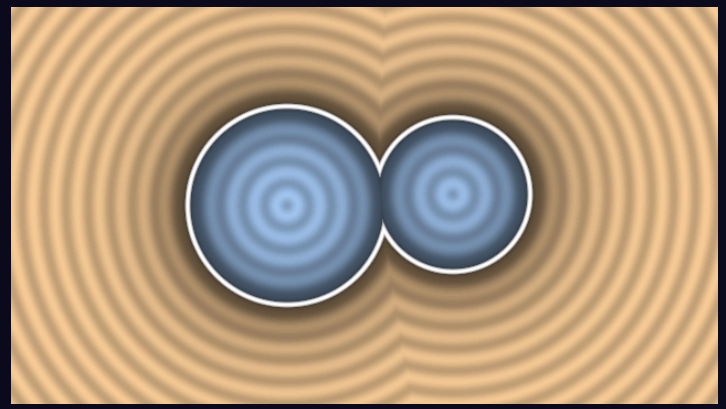


$d(p)$ is a **safe step**: nothing is closer than d, so the ray can jump that far without passing through anything (each circle touches the nearest surface, never crosses it).

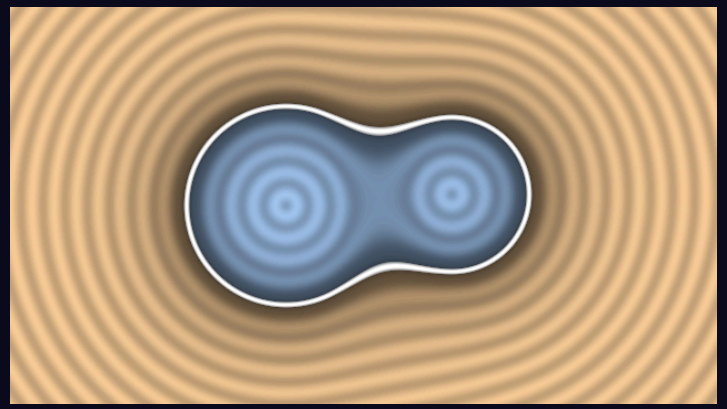
```
loop t = 0 while t < far do
  let d = scene (eye + t * dir)
  in if d < tiny then hit at t
     else t + d
```

Big steps in open space; small ones while skimming past the sphere; big again; then it closes in on the wall.

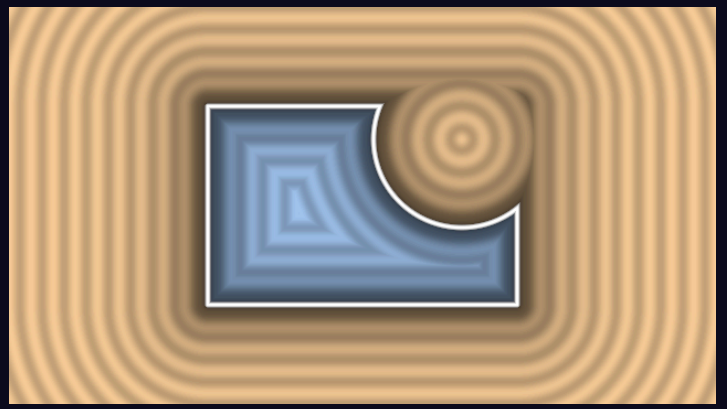
Combining shapes is arithmetic



`min a b`: both shapes (the nearer surface wins)



`smin k a b`: both, blended with a fillet of size `k`



`max a (-b)`: `a` with `b` cut out

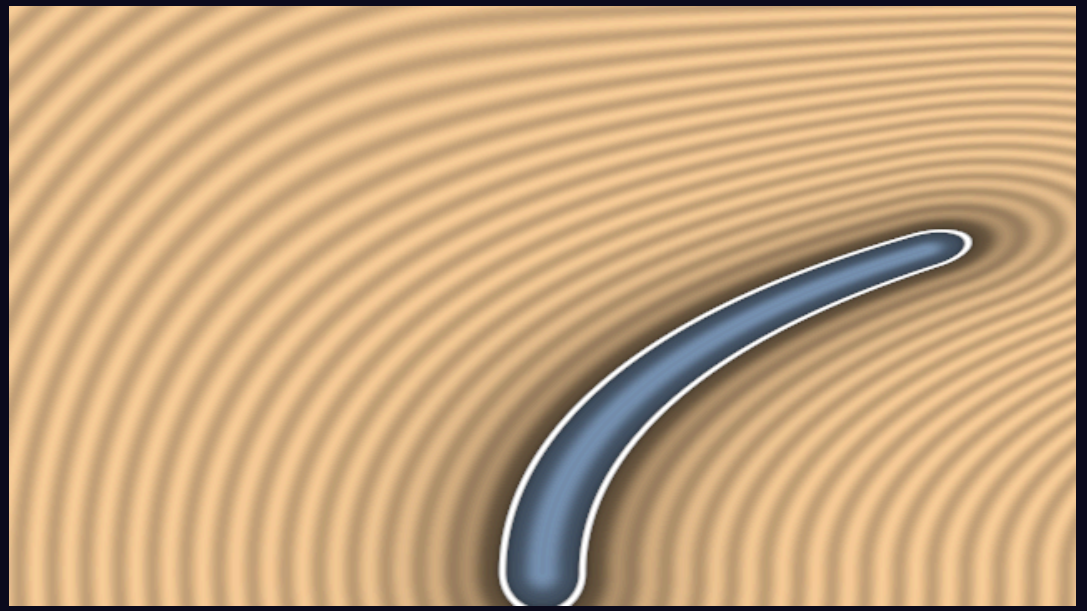
```
def smin k a b =   -- a "smooth minimum"
  let h = clamp 0 1 (0.5 + 0.5 * (b - a) / k)
  in mix b a h - k * h * (1 - h)
```

The robe flows into the shoulders, the arms into the robe, the cradle's prongs into its collar: all `smin`. Each part also says what it is made of (a **material**), so the nearest part decides the colour.

Move the point, not the shape



Repetition: before asking the circle, wrap the point into one cell (`p - cell * round (p / cell)`). Every cell then sees the same circle: infinitely many, for the price of one.



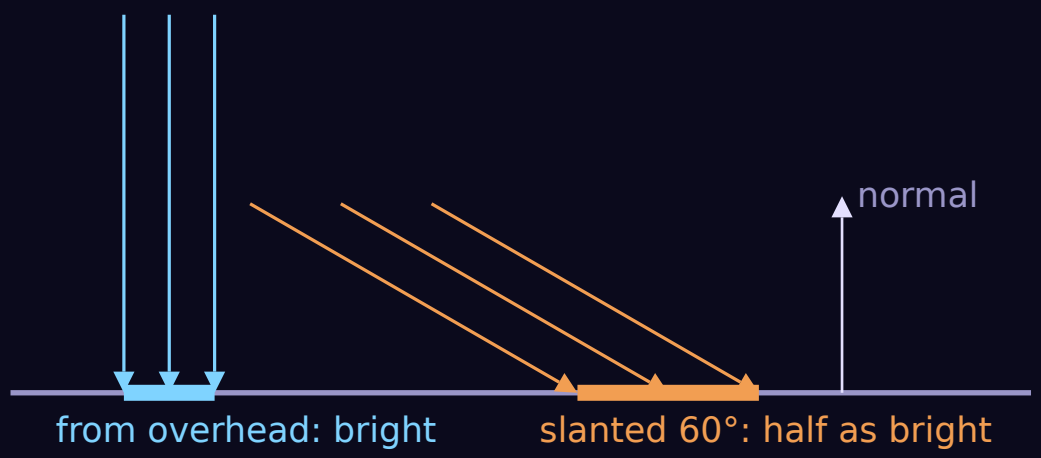
Bending: before asking an ordinary straight capsule, rotate the point by an angle that grows with its height.

The hat droops this way and the head turns this way. Bending stretches distances a little, so they are scaled down to stay safe steps.

Light

How bright is the spot a ray hits?

Light and the surface’s direction

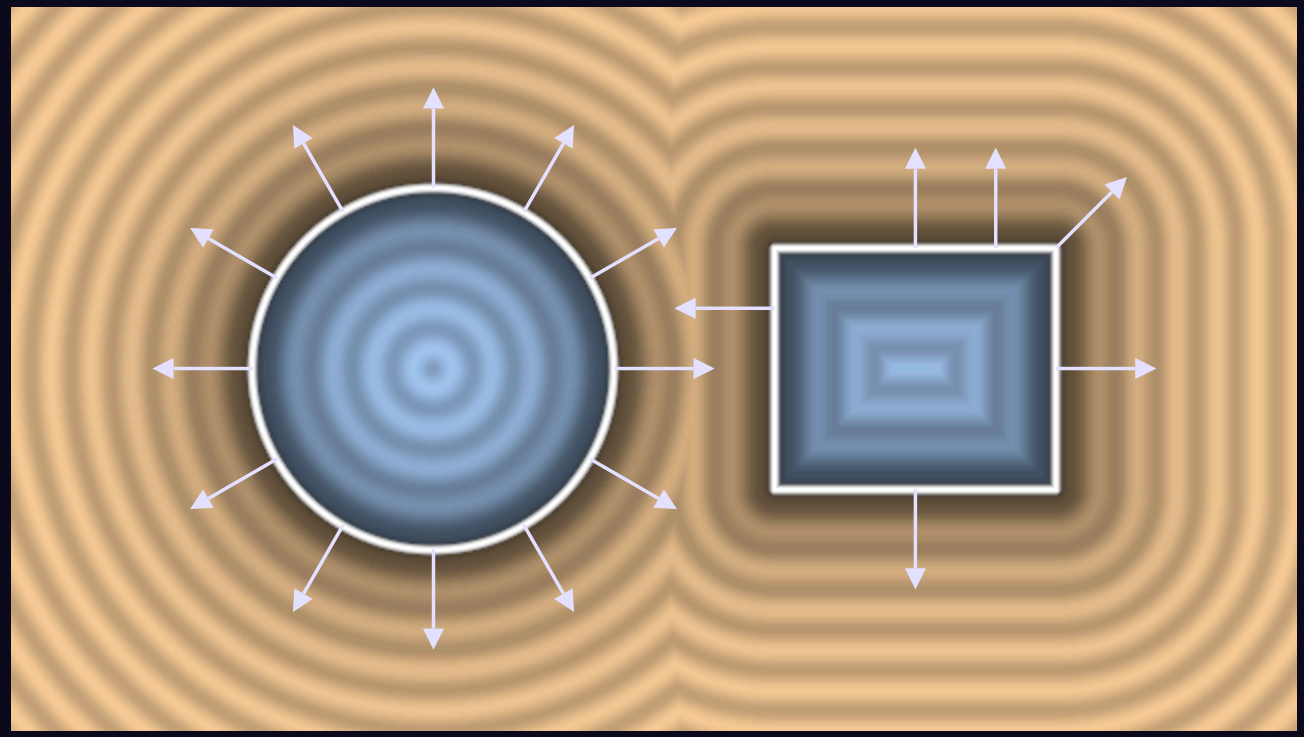


A surface’s **normal** is the direction it faces (straight out of it).

Light arriving at an angle θ from the normal is spread over a wider patch, so each spot gets $\cos\theta$ as much: the basic lighting rule.

The normal comes free from the distance function: it points the way d grows fastest (its **gradient**), found by sampling d at four points around the hit.

Normals straight from the distance function



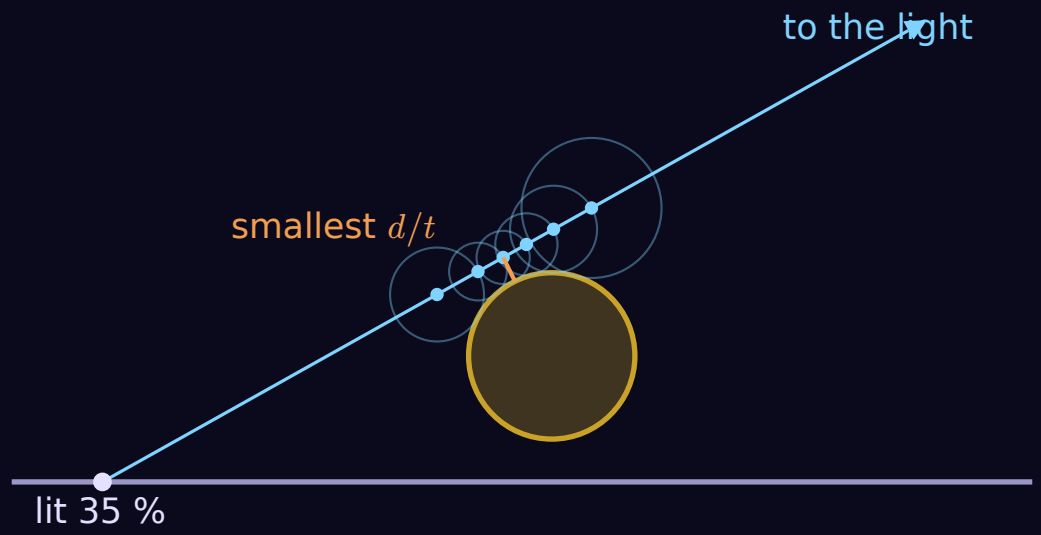
Arrows: the direction in which d increases fastest, at points on the surfaces (computed here exactly as the renderer does).

```
-- sample d a little either side of p
normal p = normalise
  (d(p + e·x) - d(p - e·x),
   d(p + e·y) - d(p - e·y),
   d(p + e·z) - d(p - e·z))
```

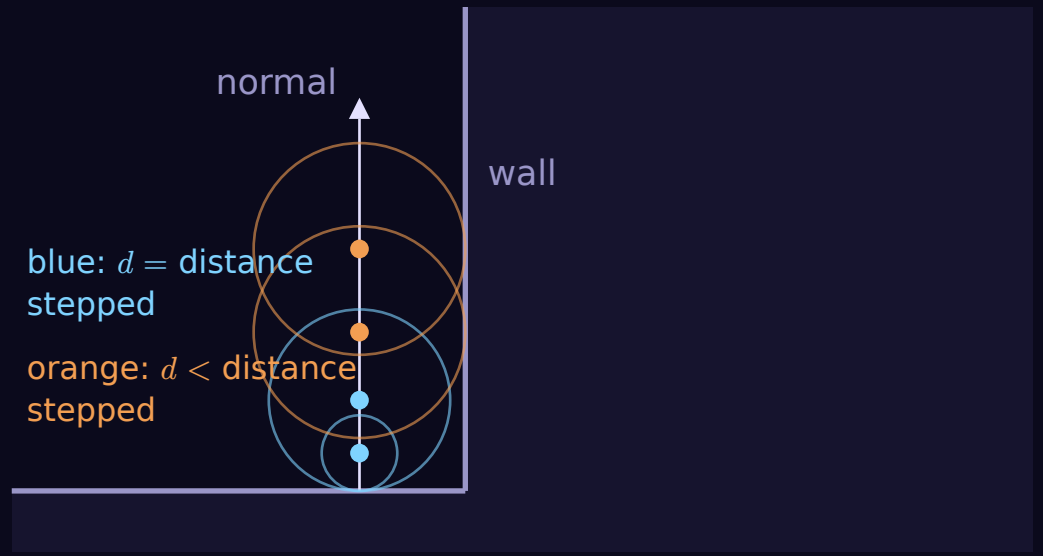
In 3D the renderer uses 4 samples on a tetrahedron rather than 6.

At a box’s corner the arrows fan out: the normal turns smoothly round it.

Shadows and dark corners, from the same function



Soft shadow: march from the surface towards the light. If the ray hits something, the spot is in shadow; if it only passes close (small d at distance t), it is partly shadowed. That gives soft shadow edges for free.



Ambient occlusion: step a little way out along the normal, a few times. Where d comes out smaller than the distance stepped, something else is close (the wall): the spot is in a corner, so less light from the sky reaches it.

Everything a pixel knows



the **normal** (direction as colour)



the **albedo**: the material's own colour



ambient occlusion: how open to the sky



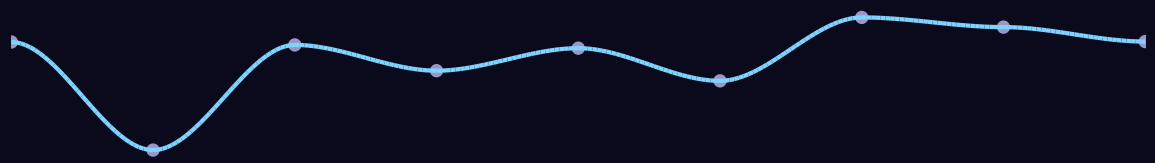
put together, for each light

The final colour: for each light (the moon, the orb, the potions), albedo × $\cos\theta$ × how unshadowed, plus a **highlight** where the light reflects towards the eye, plus a little sky light scaled by the occlusion. These are the drawer's "View" modes, used for debugging.

Detail

Patterns, glass and glow without image files

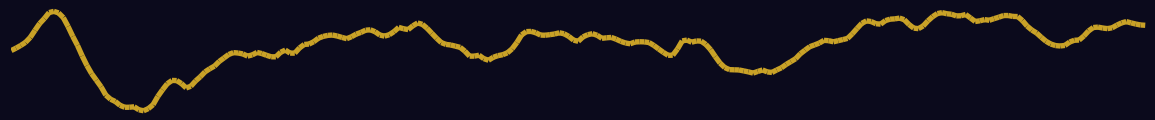
Noise: controlled randomness



value noise: a random value at each whole number, smoothly joined



the same, twice as fine and half as strong, three more times (“**octaves**”)



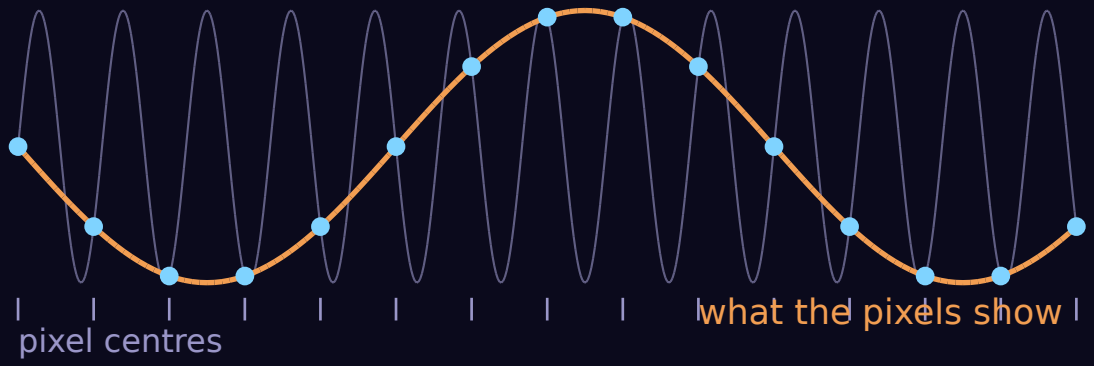
their sum (**fbm**): hills with bumps with pebbles

A **hash** turns a number into a repeatable random-looking one. **Value noise** puts hashed values on a grid and blends smoothly between them.

Adding octaves of noise gives natural detail at every scale. In 2D, that is the terrain’s height; elsewhere, the bark, cloth, stone and clouds.

It is computed, so it never runs out and needs no memory: the land goes on for ever.

Too much detail flickers



A pattern finer than the pixels (grey), sampled at each pixel's centre (dots): the samples draw a slow wave that isn't there (orange). As the camera moves, it crawls and flickers. This is **aliasing**.

A pixel at distance t covers a patch about $0.0008t$ wide (at 1080p): its **footprint**.

So every pattern fades out its details finer than the footprint before it is sampled: the bricks' joints vanish in the distance, the cloth's weave close up appears, nothing shimmers.



Patterns on surfaces



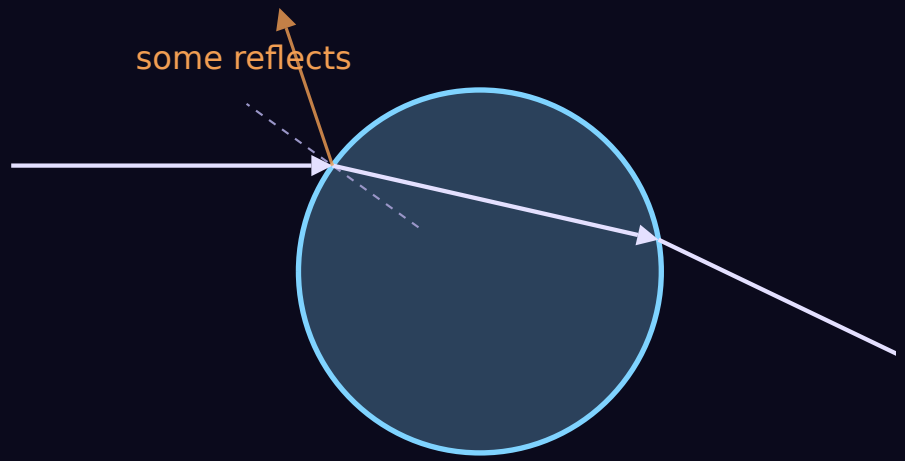
hat stars, band, eyes, brows and beard locks: shapes, hashes and noise



bricks: wall coordinates divided into cells, every other row offset by half a brick, a hash per brick for its shade

Bump mapping: a pattern can tilt the normal (by its own gradient) without changing the shape, so light catches the mortar’s grooves and the bark’s furrows as if they were carved.

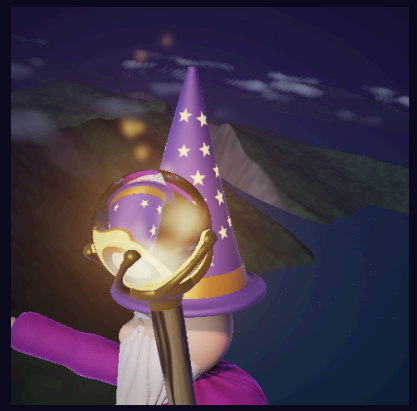
Glass and glow



the rest bends in, and out again

The orb is a glass ball, solved exactly (where a ray meets a sphere is a quadratic equation). At the glass the ray **refracts** (bends, by Snell's law) and partly reflects; the reflected share grows at grazing angles (**Fresnel**).


lightning


fire

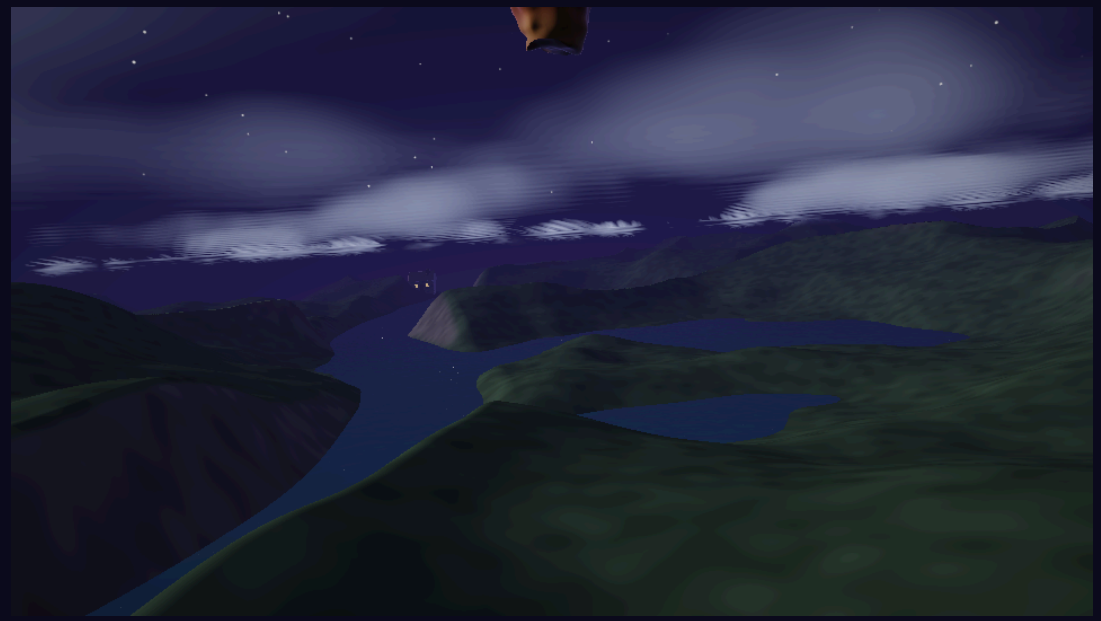

wind

Glows are added along the march: near the orb, each step adds a little light, more the closer it passes.

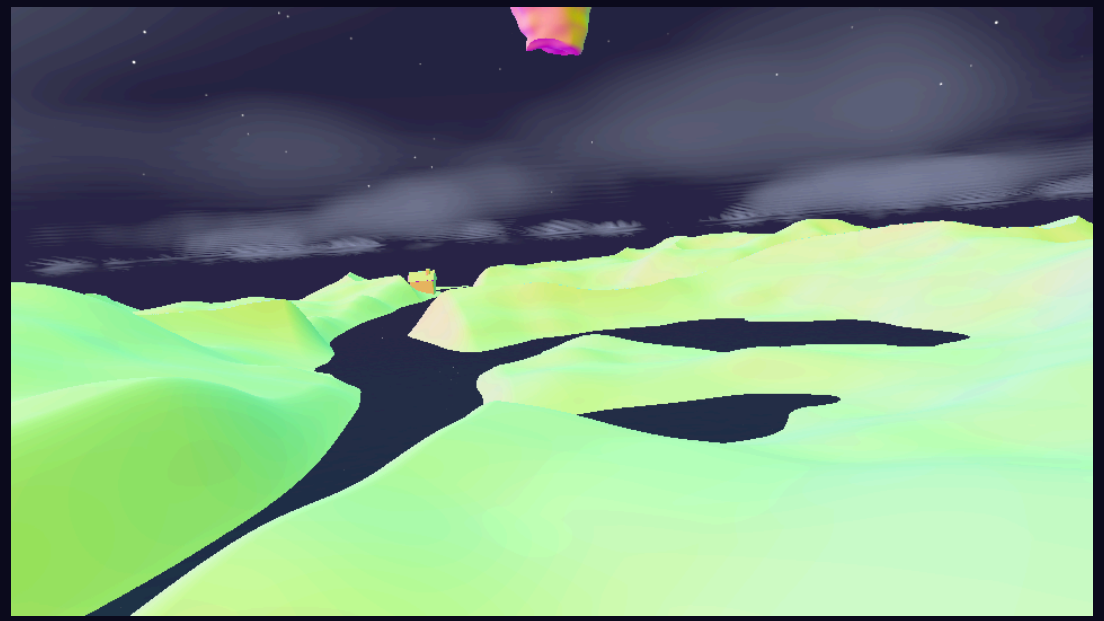
The world

Endless land, and keeping the marching fast

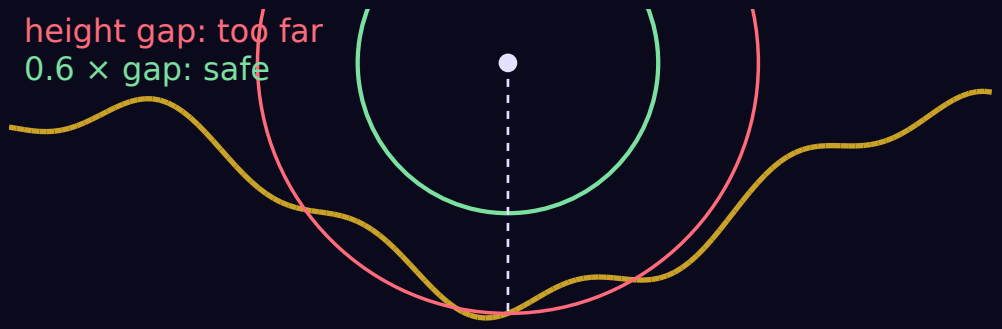
Terrain: a height map is not a distance



the land: fbm heights, ridges, rivers carved in, a sea

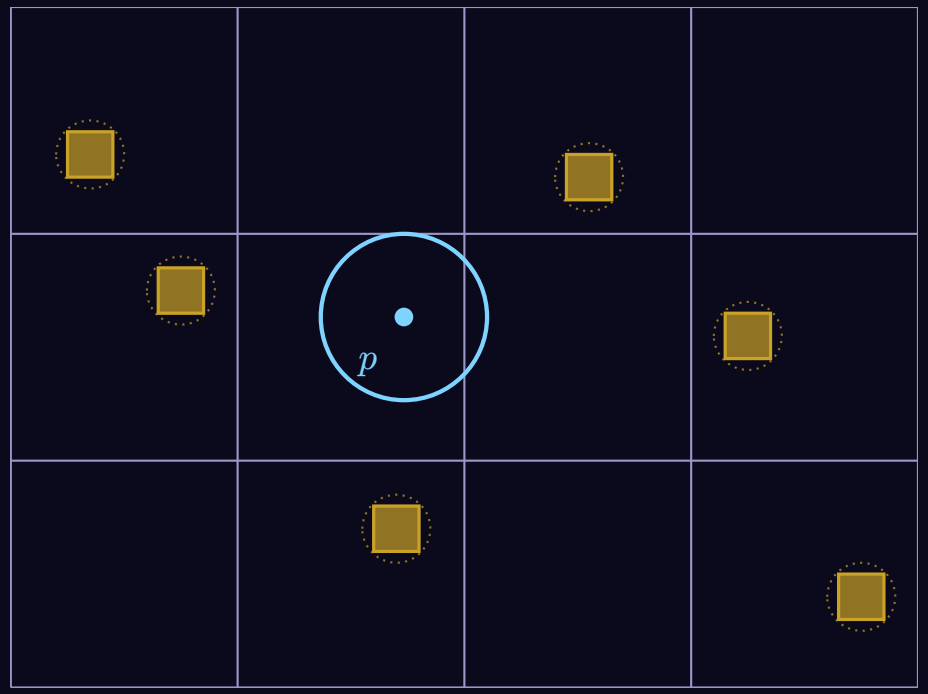


its normals, from the noise’s exact slope (differentiated with it)



The height above the ground (dashed) is **more** than the distance to a slope: stepping that far could jump into a hillside (red). Because the slopes are never steeper than about 53°, 0.6 of it is always safe (green).

Endless buildings from a grid



Each cell’s hash decides: a building or not, which kind, where (kept a margin in from the edges) and how turned.

Checking every building would be impossible: there are infinitely many. Instead, the distance function returns a **lower bound**: a step that cannot reach any building.

A building in another cell is at least as far as this cell’s border, plus the margin, less its own size. That is the blue circle: the ray at p may step that far.

Every shortcut in the renderer works like this: it may underestimate the distance (costing an extra step) but must never overestimate it (that would step through a wall).

Two marches per ray

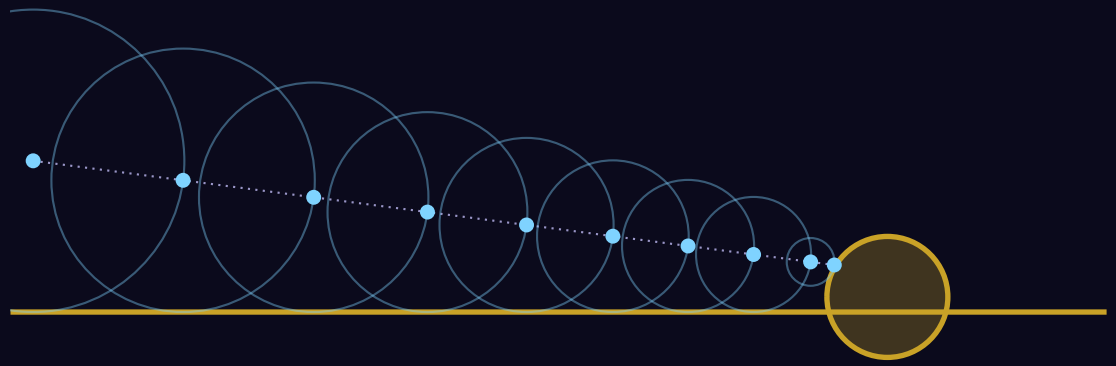


The scene mixes millimetre fingers with a 150-unit horizon. One march over everything would take tiny steps all the way.

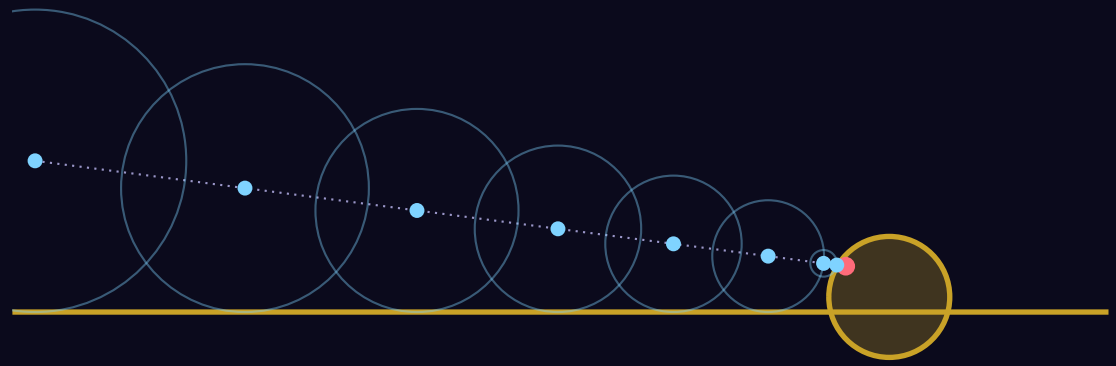
So each ray is marched twice: through the **land** along its whole length, and through the **wizard, rock and glows** only inside a sphere around him. The nearer hit wins.

Every part also has a **layer** bit, so one scene function serves the camera, the shadows (only some parts cast them) and debugging toggles.

Taking bigger steps, and backing off



safe steps: **10** evaluations



steps of 1.4 × d: **9** evaluations; red: overshot, so it backs off

Near a flat surface, the safe step shrinks in proportion to the distance, so long grazing rays creep. Stepping 1.4× further is usually still safe; the next point's own distance shows when it wasn't (the two circles no longer overlap), and the march steps back. Keinert et al., "Enhanced Sphere Tracing", 2014.

Engineering

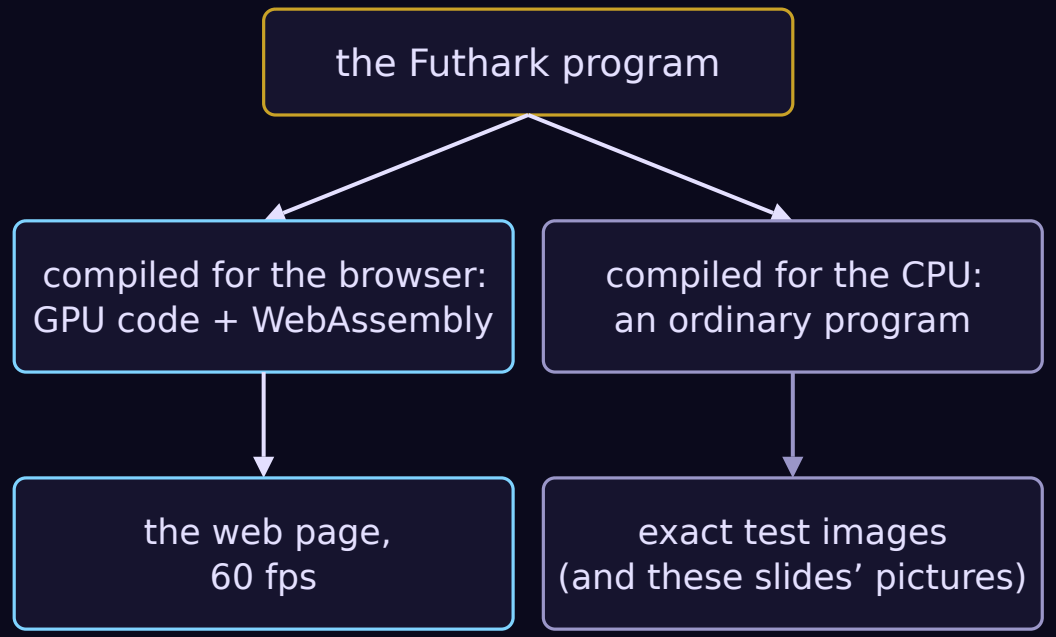
Futhark, the browser, sound, and speed

Futhark

A pure, statically typed **array language** (think of a small ML or Haskell). You write `map`, `reduce`, `scan`, `filter` over arrays; the compiler turns them into GPU kernels.

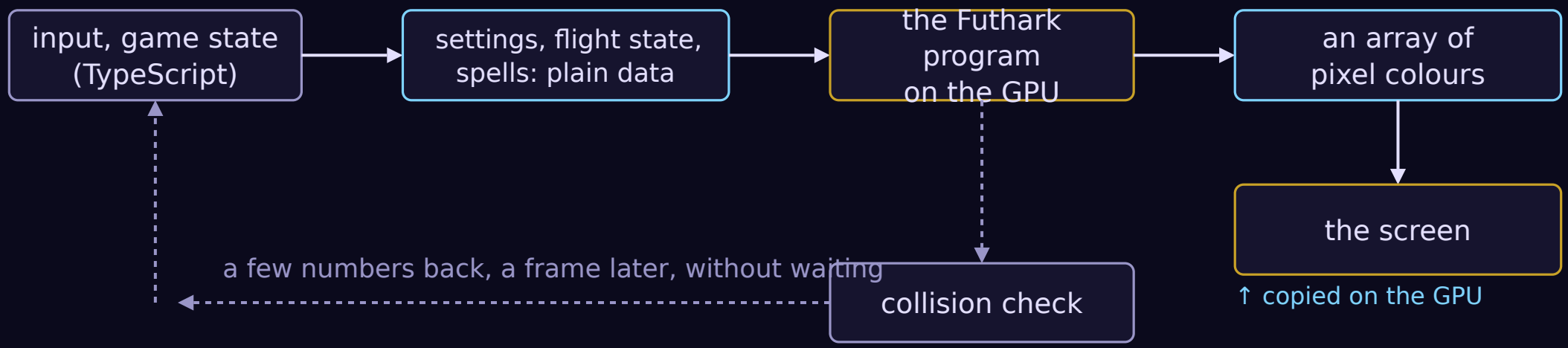
```
-- one frame: a colour for every pixel
def render_frame prm w h ... : []u32 =
  tabulate (w * h) (\i ->
    let (x, y) = (i % w, i / w)
    in pack (pixel prm x y))
```

No GPU code is written by hand.



One source, two targets: every refactoring is checked to change no pixel.

From the GPU to the screen



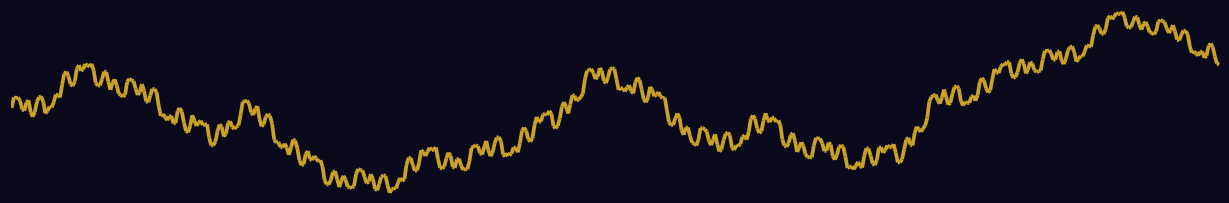
The picture never leaves the GPU: the output array is copied straight into the screen's image. Reading results back to the page is slow and stalls, so only a few numbers come back (for collisions), and only when ready.

The renderer keeps no state: everything that changes (camera, flight, spells) lives in the page and is passed in each frame, so the same inputs always give the same picture.

Music from a function of time



one second of the music (left channel)



12 ms of it: 576 samples, each computed from its time alone

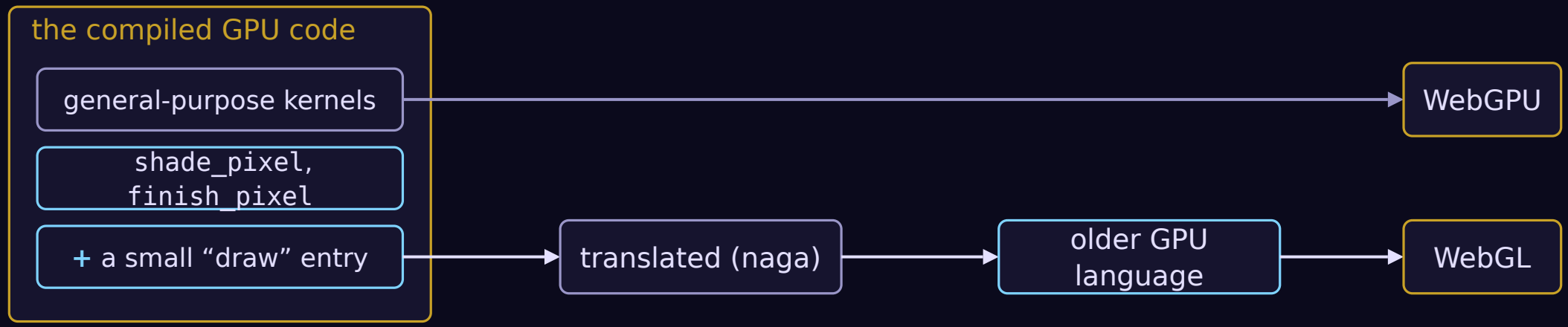
Sound is a list of numbers (**samples**), 48 000 a second per ear.

As in **Shadertoy**, each sample is a pure function of time: `music t = (left, right)`. Notes, chords, echoes and drums are all formulas in t.

```
def sound t0 rate n =  -- a chunk
  tabulate n (\i ->
    music (t0 + i / rate))
```

The page asks the GPU for ≈ 85 ms of samples at a time, keeping ≈ ⅓ s queued ahead of the speakers. Time stays in double precision on the page: in 32-bit floats it stops resolving single samples after a few minutes.

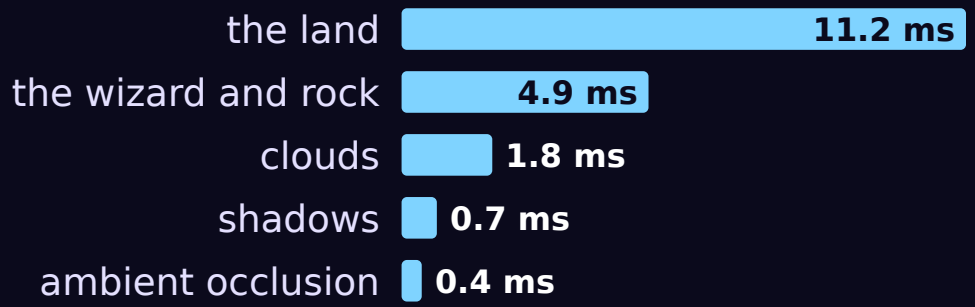
A fallback for browsers without WebGPU



WebGPU (the modern browser GPU interface) runs general-purpose kernels; the older WebGL can only colour pixels. The per-pixel work is in two functions, so each build also gets a small entry point that just colours a pixel with them, translated for WebGL. Spells, collisions, anti-aliasing and music need the general kernels, so the fallback does without.

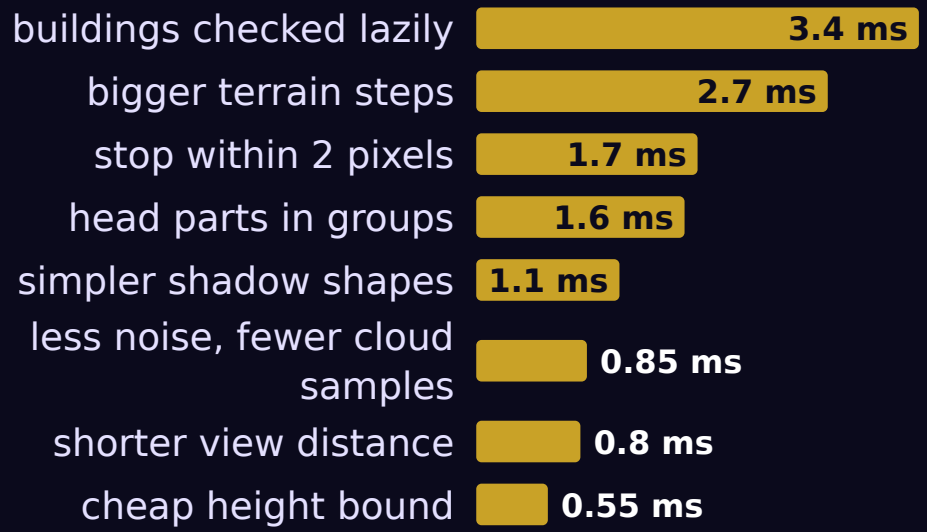
Measuring where the time goes

Switch one part off, and time it (milliseconds per frame; 16.7 ms is the budget for 60 fps):

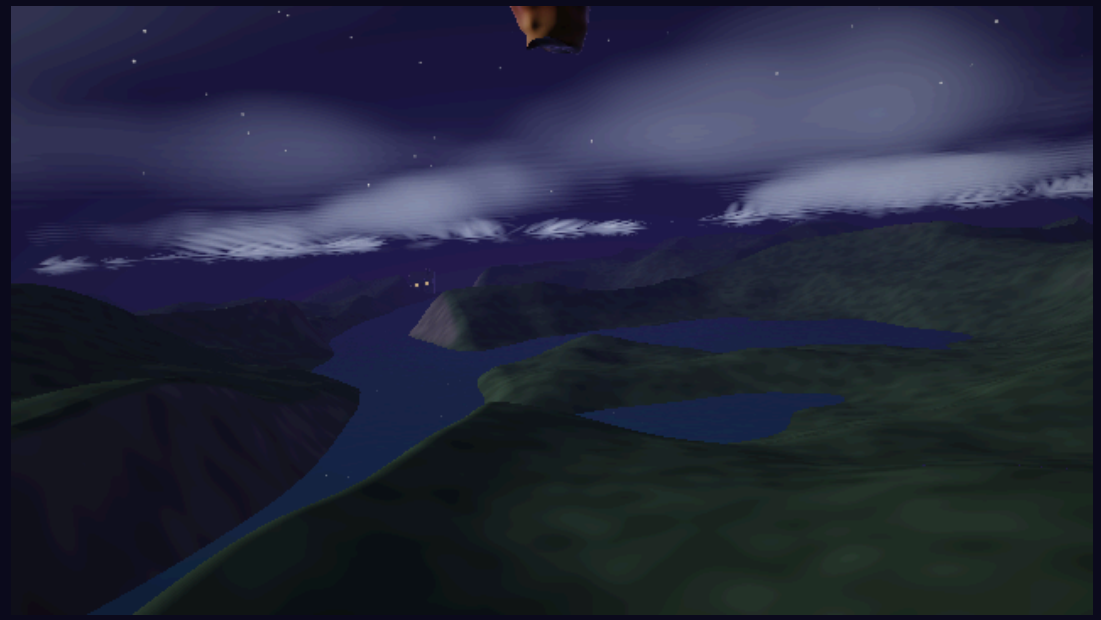


Measured at 18.9 ms per frame, before the last round of work.

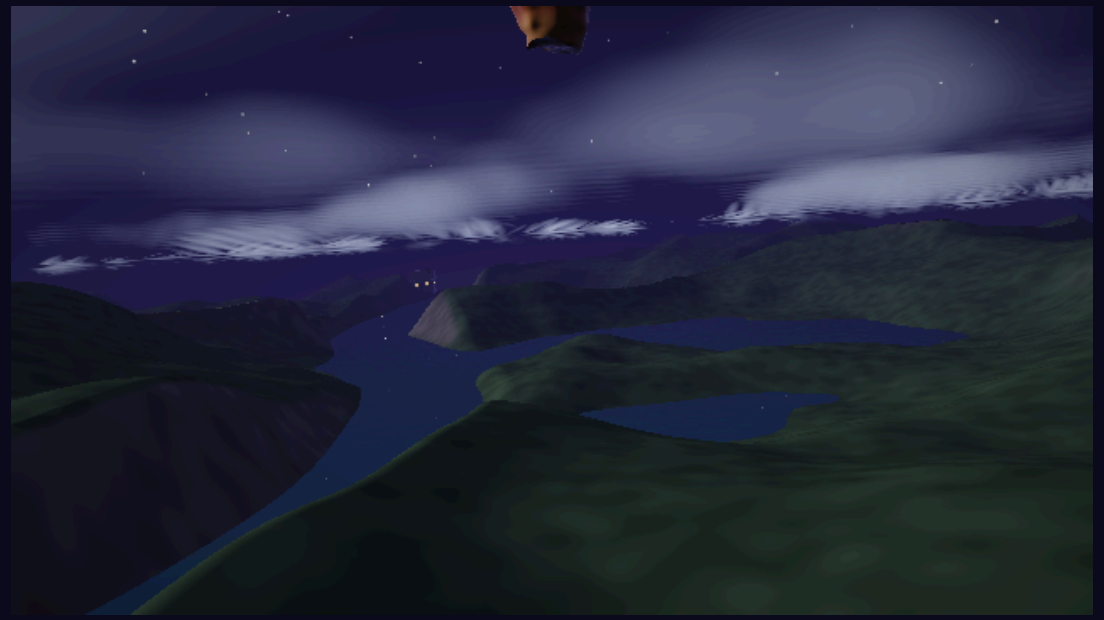
What saved the most, ms per frame:



Good enough is a pixel



the march stops within $0.0004t$ of the ground



within $0.0015t$ (2 pixels' depth): 1.7 ms faster

A pixel at distance t is about $0.0008t$ wide, so finding the ground any closer than a pixel or two changes nothing you can see: no outline moves. Matching the precision to the pixel size was the cheapest win.

Smaller GPU code loads faster



The compiler copies every function into every place it is used (**inlining**). The GPU code reached 2.4 MB, and the browser takes seconds to compile it.

Keeping a function separate (`#[noinline]`) means passing it its inputs: the wizard's whole pose is ≈ 200 numbers, too many to pass around.

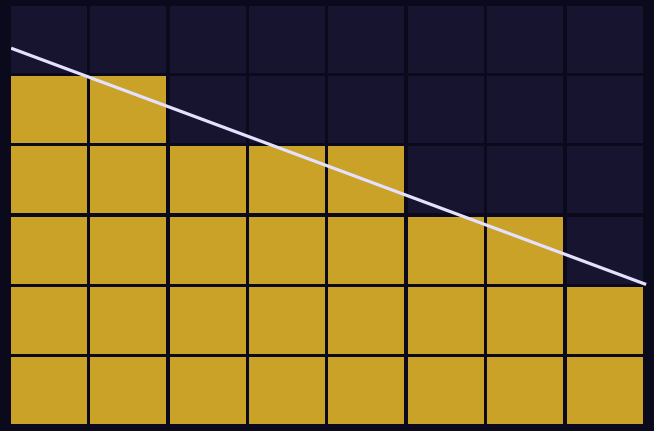
```
-- the 10 values the sky needs, not 200
type sky_in = { horizon, zenith, moon,
                moon_size, ..., secs }

#[noinline]   -- one shared copy
def sky_colour (sk: sky_in) dir = ...

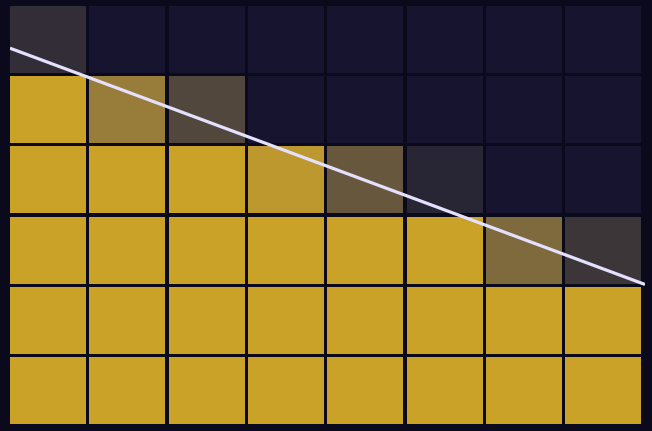
-- the callers are unchanged
def sky pose dir =
  sky_colour (sky_in_of pose) dir
```

Plus fewer separate GPU programs to compile. Time to the first frame roughly halved.

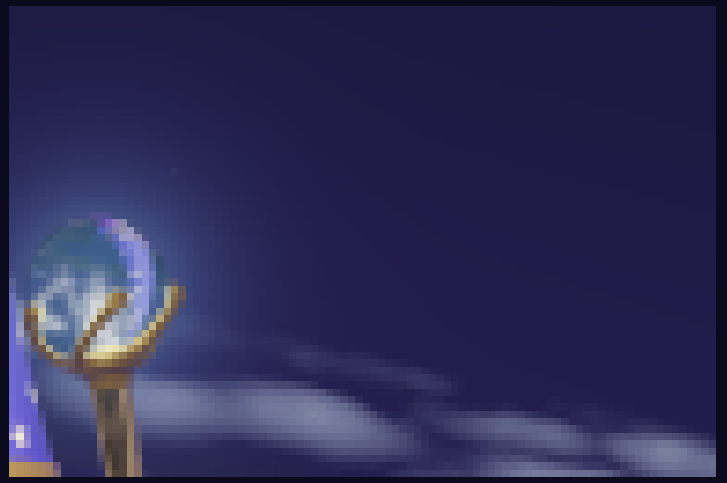
Smoothing jagged edges



one ray per pixel: each pixel all-or-nothing, a staircase



16 rays per pixel, averaged: each pixel by how much of it is covered

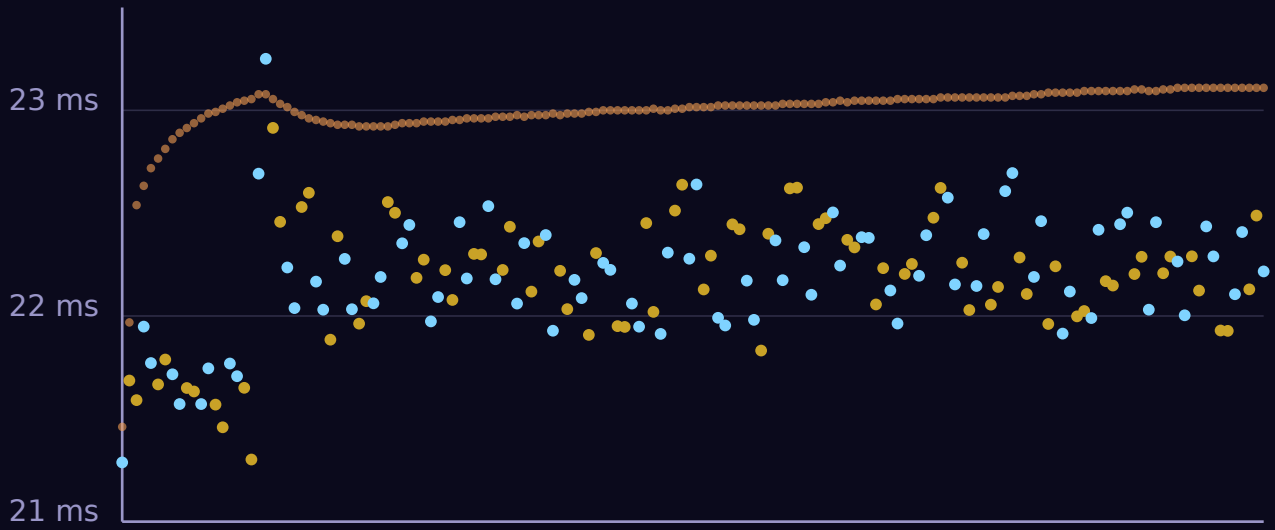


rendered: several rays where edges are

Tracing extra rays everywhere would cost 4–16× as much. The renderer finds the pixels on edges (where neighbours differ a lot), gathers them into a list, and traces 4 more rays for those only.

Measuring honestly on a laptop

frame time of two **identical** builds (blue, gold); CPU °C (orange, 50–90)



Comparing a build with **itself** shows the noise: frames get ≈ 1 ms slower over the first minute as the chip heats, then settle. Fans at full speed changed nothing: the laptop is limited by power, not heat.

So: warm up first, alternate the builds A B B A, and only believe a difference larger than its 95% confidence interval.

to detect	rounds needed
0.5 ms	≈ 3
0.25 ms	≈ 9
0.1 ms	≈ 55

Bugs found along the way

- **false arriving as true**: a 1-byte flag copied into a 4-byte slot of uncleared memory (patched)
- **function names containing “z”** broke calls between GPU functions (fixed upstream)
- `x ** 2` **with negative** `x` is “not a number” on the GPU: one explosion ring turned the whole screen white
- **an empty array** bound as a too-small GPU buffer
- **a 4 KB save area** overflowing when the page paused mid-frame to wait for the GPU
- **the browser compiling GPU code on its main thread**, freezing the page for seconds

Most were found by comparing the CPU and GPU pictures, or by building variants to narrow a problem down.

Takeaways

- **One distance function** gives shapes, normals, shadows, occlusion and collisions
- **Bounds that never overestimate** are what make it fast: a shortcut may waste a step, never skip a wall
- **Pure functions** of time and input are easy to test exactly, for pictures and for sound alike
- On the GPU, **code size** matters as much as arithmetic
- **Measure**, and know how noisy the measurement is



github.com/badly-drawn-wizards/wizard-vibes